

Typed Memory for Autonomous Agents: Co-Locating Schema, Instruction and Substrate in One Authored Artifact

Joachim Rosskopf

Abstract—Retrieval-augmented agent memory is commonly organized as a vector index over unstructured text, which offers no level of description between the individual chunk and the whole corpus. Typed alternatives are not new: semantic wikis, knowledge graphs and ontology-based data access have supplied schemas, typed links and federated bindings for two decades. This paper argues that what an agent needs is narrower and specific: the schema a record must satisfy, the instruction a model executes when handling records of that kind, and the declaration of where those records physically live, should be *one authored, versioned artifact* rather than three maintained separately. We call such an artifact a *skill*, and develop the consequences. Discovery then enumerates authored types rather than generated records; the same artifact that a curator edits is the one a model reads; and a type library ships with its integrations, because the substrate binding is a field of the type. State is maintained by projecting an immutable event log through agent runs whose instructions are themselves such artifacts. We report a 101,500-line implementation and three feasibility results: type enumeration costs a fixed 744 tokens and sub-millisecond latency as the corpus grows a thousand-fold; one navigation sequence reaches five heterogeneous substrates from known handles; and event projection produced no duplicate write across 35 process kills landing mid-write. These establish that the arrangement is buildable and cheap to offer. They do not establish that agents exploit it, which requires multi-type benchmarks with type-naming queries that do not presently exist, and which we argue is the field’s most useful next contribution.

Index Terms—Autonomous agents, agent memory, knowledge representation, retrieval-augmented generation, large language models, typed data models, event sourcing.

I. INTRODUCTION

STORING a million facts has not been difficult for decades. Placing the right handful of them into a bounded context window, repeatedly and cheaply enough that an agent can afford to look, is what constrains autonomous systems today. Every retrieval a language model performs is paid for twice, in tokens and in turns,

and both prices follow from one design decision: how much the memory makes the agent read before it can decide what to read next.

The most common answer is a vector index over chunked text. It answers one question well—“what text resembles this text”—and charges a document body for every answer. Structured alternatives exist and predate the current interest in agents: knowledge graphs [1], semantic wikis [2], collaboratively curated typed entities [3], [4], and tool schemas that constrain what a model may call [5]. Our argument is therefore not that structure is missing from the field, but that the structure agents need is distributed across artifacts that are maintained separately: a schema in one place, the natural-language guidance a model follows in another, and the mapping to where data physically resides in a third. Each is authored by different people, versioned on different cycles, and reviewed by different processes.

Thesis. Agent memory should be *typed*; the types should be authored by people and readable as instructions by models; and state should be derived from an immutable event log by agent runs whose instructions are themselves part of the typed corpus. Two dualities express this, and their composition is the contribution.

Skill $\leftrightarrow$ *Instance*. A skill page is at once the schema a class of records must satisfy and the prose an agent reads to learn what that class is for. An instance is at once a record of that type and a document a person can edit. Discovery then operates over *types*, which are authored and therefore bounded in number, rather than over *records*, which are generated and therefore not.

Event $\leftrightarrow$ *Callback*. An event is an immutable fact; a callback is an agent run that folds it into instance state, and the run’s instructions are the event’s own skill page. State is a projection of the log, so it is re-derivable, and an interrupted projection is replayable rather than corrupting.

A corollary with disproportionate practical weight. Because the *skill* declares its storage backend, the substrate beneath an instance is a property of its type rather than of the referent space. One type’s instances are documents; another’s are rows of a read-only relational view; another’s are chunks extracted from an uploaded file; another’s are fetched live from a remote service on each read and accept writes back upstream. The agent reaches all of them with the same navigation verbs. This is what

J. Rosskopf is with DataZoo GmbH, Germany (e-mail: jr@data-zoo.de).

Figures marked x^P are prototype-era, from a 430-line Python predecessor and its verification record; they are never presented as results of the system described here. All other figures were produced by harnesses in the accompanying repository.

allows a typed memory to reason over an organization’s data without first importing it.

Contributions.

- 1) A memory model for autonomous agents built on two dualities and a substrate corollary, with an explicit argument for why typing—not disclosure discipline alone—is what bounds context cost (Sections II–III).
- 2) A realization of the model in 23 Rust crates and 101,500 lines, including the mechanisms it requires: index-time reference validation, a single embedded store for the index concerns, five instance backends behind one referent space, and a projection loop whose reducer generalizes from single-hop cascades to multi-phase fan-out (Section V).
- 3) A guarantee register of eighteen safety and semantic properties in which every row names an integration test that executes in the project’s ordinary test run (Section VI).
- 4) Feasibility evidence for three properties of the arrangement, bounded by an explicit account of what the measurements do not reach (Sections VII–VIII).

II. WHY UNTYPED MEMORY DOES NOT SCALE

A. A Cost Model

An agent’s cost on a task is the tokens carried in context multiplied by the turns taken, and both follow from how much the memory requires it to read before acting. Following the progressive-disclosure principle introduced for procedural knowledge [6], we distinguish *Tier 1*, the metadata loaded at cold start, from *Tier 2*, the bodies subsequently retrieved.

B. A Chunk Index Has a Weak Tier 1

Tier 1 requires something enumerable and small. A chunk index can offer collections, namespaces, directory structure, tags or generated summaries, so it is not literally without metadata; but none of these is a *contract* that records are validated against, and none carries guidance a model can act on. What an agent can enumerate is therefore either too coarse to select on or too numerous to hold. A vocabulary of authored types supplies both properties at once: it is small because people write it, and it is actionable because each entry states what records of that kind are for.

Table I gives the resulting arithmetic, measured on the predecessor system’s corpora. The absolute figures are prototype-era; the ratio is the point. At ten thousand pages, reading metadata alone costs three to five thousand tokens and reading every page costs four to six hundred thousand, while enumerating *types* costs a constant independent of the corpus.

C. Where the Binding Lives

A vector index reasons only over text already ingested, so tables, archived files and records behind service interfaces must be copied in and chunked; the copy is

TABLE I
COLD-START COST BY DISCLOSURE STRATEGY. PROTOTYPE-ERA MEASUREMENTS, TIKTOKEN CL100K_BASE, AGAINST A NOMINAL 30K WORKING BUDGET.

strategy	10k pages	100k pages
every page, full content	400–600k ^P	4–6M ^P
every page, metadata only	3.2–4.8k ^P	8–12k ^P
every <i>type</i> , metadata only	189 ^P	189 ^P

stale on arrival and the refresh pipeline is a component somebody operates. Federated approaches solve exactly this. Ontology-based data access and virtual knowledge graphs [7] expose remote sources through a common vocabulary without copying, as do dataspace [8] and linked data [9]. Our claim is not that federation is missing but that the *mapping* is conventionally an engineering artifact held apart from the vocabulary—authored in a mapping language, deployed by a different team, invisible to whatever instructs the consumer. Placing the binding in the type changes what can be distributed: a vocabulary and its integrations become one versioned object, discussed in Section V-E.

D. Longer Contexts Do Not Resolve It

Large context windows make the naive strategy feasible at small scale and still inadvisable. Cost and latency grow with the window, and retrieval accuracy degrades for material in its middle [10]. More fundamentally the strategy has no stable operating point: a corpus that fits today does not fit next year, and the failure is silent.

E. Writes Without Schemas Accumulate Contradictions

The write path of an untyped memory is “embed and append.” Nothing validates new material, marks what it supersedes, or links it to the entities it concerns. Contradictions accumulate at exactly the rate content does, and a layer that ranks by similarity cannot prefer the correct one of two similar answers.

F. Typed Memory Without a Loop Goes Stale

The complement is equally damaging. A typed knowledge base with no ingestion loop is a wiki: correct when written, quietly wrong some weeks later, and trusted considerably longer. A model of agent memory that stops at the data model has described half a system, which is why the model below has two dualities rather than one.

III. THE TWO-DUALITY MODEL

A. Skill ↔ Instance

The starting observation is structural: an agent-skill page and a typed wiki note are the same artifact—text with structured frontmatter, an identifier and a description—differing only in whether the reader treats the content as instructions or as data. Letting an instance declare which skill it conforms to makes that skill its type, and makes the instruction and the schema the same page.

A corpus is a set of pages $P = S \cup I$ with S the skills and I the instances, a total function $\tau : I \rightarrow S$ given by each instance’s declared type, and a typed reference relation $R \subseteq P \times S \times P$ in which $(p, s, q) \in R$ requires $\tau(q) = s$. The requirement is checked when a page is indexed rather than when a reference is followed, which makes a dangling or mistyped reference a reportable error instead of a runtime surprise.

Why this bounds context cost. Discovery enumerates S , not I . S is authored by people—it is the vocabulary of a domain, not its contents—so its cardinality is bounded by human attention rather than by ingestion rate. A tenant ingesting a million meeting records still has one **meeting** type. The precision matters: progressive disclosure bounds cost *only because* typing supplies a level of description at which to stop. Applied to an untyped corpus, disclosure has nothing to disclose about.

Concretely, an agent asked what changed for a customer last quarter first enumerates types, receiving on the order of a hundred identifier-description pairs and no bodies. It selects two, issues one listing filtered by type and date range, and expands the two or three records it needs. The remaining records cost nothing, because no step in that sequence has a size depending on how many exist.

B. References Carry Type, Locus and Version

A reference splits on alias, anchor, version and type separators, yielding six usable forms from a bare type reference to `[[type::id#anchor@version|alias]]`. The anchor addresses a block within a record; the version pins a snapshot, so a reference into external versioned data denotes the same bytes indefinitely. Typing also removes an ambiguity untyped wiki references leave open: in the predecessor, `[[Anna Mueller]]` and `[[anna-mueller]]` resolved to two distinct nodes because nothing defined a canonical form.

C. The Corollary: the Type Declares the Substrate

Skills carry a backend declaration, and it is the skill that carries it—never the individual instance, never the caller. Five substrates are realized here: native documents, a read-only relational view over an external database, an extracted and chunked binary file, a live remote service, and a live upstream tool server. The last two hold no local copy: the body is fetched on each read and edits are forwarded upstream.

One invariant makes this inexpensive. Every external instance retains a native overlay page, which *is* the instance as far as the referent space is concerned—identity, references, access control and history reuse existing machinery—while a binding record connects it to the external data. All novelty is confined to where the body originates, which is why adding a substrate required no change to the dispatcher, the wire format, or the reading agent’s vocabulary.

One division shows through, and we state it rather than claim a transparency the model does not have: materialized substrates are indexed and therefore reachable by

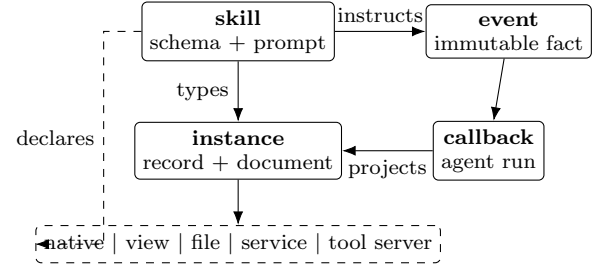


Fig. 1. The model. A skill types its instances and instructs the callback handling events of that kind; an event fires a callback, which projects it into instance state. The substrate beneath an instance is selected by the *skill*, never by the caller.

similarity, whereas live proxies are not. A remote instance is reachable by reference, not by resemblance.

D. Event $\leftrightarrow$ Callback

An event is an instance of an event-typed skill carrying a timestamp and typed references to what it concerns. Events are immutable by convention: a correction is a new event citing the original, not an edit. The inbox is the unprojected suffix of the log.

When an event arrives, a runner triggers an agent harness whose task description *is* the event’s labelling skill page and whose tool surface is the ordinary agent API. There is no per-harness client and no prompt template maintained outside the corpus: the instructions for processing a class of event live in that class’s type, where they are read, reviewed and versioned like any other page.

State therefore lives in the knowledge base rather than in an orchestrator’s variables. This inversion is what makes the loop recoverable: an engine holding progress in local memory loses it when the process dies, whereas a projection over a durable log resumes by re-reading what it already wrote.

E. Composition

The dualities are not independent contributions sharing an implementation; each supplies something the other requires. The skill-as-schema is what a callback’s write is validated against, and the skill-as-prompt is what drives the callback at all. The event is an instance’s provenance; the callback is how the instance acquires a current value. The substrate corollary lets the loop reach beyond the corpus: because an instance may be a live remote record, a callback can fold an event into a system the memory does not own, writing through rather than into a copy that will drift.

Remove the corollary and the model still functions, but only over material already imported—which is where most retrieval-augmented architectures sit, and why so much of their engineering is pipeline engineering.

IV. WHAT IS AND IS NOT NEW HERE

Each element of the model has precedent, and it is worth stating plainly which. Typed records with human-authored

schemas and typed links are the substance of semantic wikis [2] and of collaboratively curated entity bases [3], [4]. Exposing heterogeneous sources through one vocabulary without copying is ontology-based data access and virtual knowledge graphs [7], anticipated by dataspace [8] and linked data [9]. Deriving state by folding an immutable log is event sourcing [11]. Constraining what a model may invoke by a schema is standard tool use [5]. None of these is claimed as a contribution.

The claim is about *co-location*. In the systems above, three artifacts are maintained separately: the schema a record satisfies, the guidance a consumer follows when handling records of that kind, and the mapping from the vocabulary to physical storage. An ontology does not tell an agent what to do with an instance; a mapping file is invisible to whatever does; and the prose that guides a human curator lives in a manual. This paper’s position is that for agent memory the three should be one authored page, and that three consequences follow which do not follow from any of them separately.

Curation and instruction become the same act. Editing what a type means edits what the agent does with it, in the same review, with the same version history. There is no drift between an ontology and the prompt that consumes it, because there is no prompt outside the corpus.

Discovery and specification become the same enumeration. The set an agent lists at cold start is the set a curator maintains. Its cardinality is therefore governed by human attention rather than by ingestion rate—the property Section VII measures—and it cannot silently diverge from the schema, because it is the schema.

Vocabulary and integration become one distributable object. Because the substrate binding is a field of the type, a type library ships with the means of resolving its instances against a subscriber’s own systems (Section V-E). Ontology-based access separates these by design, which is appropriate when the ontology is a shared standard and awkward when the vocabulary is a working artifact of one organization.

A reasonable objection is that co-location is an implementation convenience: a callback could equally be driven by code, a policy file or a tool schema outside the corpus. That is true, and it is why we do not claim the arrangement is necessary. We claim it is consequential, and Section VII reports what it costs to offer; whether it changes agent behaviour is the open question of Section VIII.

V. REALIZATION

A model of this kind is interesting only if it can be built without the mechanisms quietly reintroducing the costs it claims to remove. We describe an implementation, in 23 Rust crates and approximately 101,500 lines, that realizes each element.

A. One Embedded Store for the Index Concerns

Each tenant is a document directory beside a single embedded analytical database file [12]. That file carries

relational metadata and the typed reference graph; a block table whose vector column is indexed by HNSW and whose text column by BM25; the operation log of the collaborative editing layer; and an attachment point for read-only lakehouse catalogs [13]. A type-filtered vector query is consequently a single SQL statement, because the filter columns are denormalized alongside the vector.

The document directory remains canonical and everything in the database is derivable from it. This keeps a typed memory inspectable by ordinary tools: version control still diffs the corpus, text editors still edit it, and recovery is a rebuild rather than a restore.

B. Validation at Index Time

Reference parsing is by regular expression rather than an AST walk, because document parsers split reference delimiters across nodes in ways depending on surrounding context. Four checks run in the same pass—the type exists, the target exists, the target’s declared type matches the reference, and any named anchor is present—and their output is the same issue list the validation API returns for a dry run, so an authoring agent receives exactly the feedback the indexer would give, before it writes.

C. One Referent Space, Five Substrates

The corollary is where an implementation makes outbound network calls on behalf of tenant-authored content, so its safety properties belong to the design rather than to operations. Credentials are registered out of band and never appear in the corpus. A remote type names an administrator-registered endpoint rather than a URL, so tenant-authored text cannot direct the system at an arbitrary host, and a declared kind disagreeing with the registered kind fails closed. A read that times out returns the overlay page with an explicit issue marker rather than a partial body. Write-back templates are bound by value, and an unresolvable placeholder aborts the call rather than transmitting a literal. A relational binding records a fingerprint of its source schema and revalidates it, marking the binding degraded when the source has moved beneath it.

D. The Projection Loop

All automation resides outside the knowledge base. A separate runner subscribes to an outbound notification and also polls the inbox—the poller being the self-healing path for a missed notification—and both converge on one bounded queue per tenant. A durable ledger, deliberately outside the tenant store, records one row per run and is the basis of idempotency, deduplication, depth and budget limits, and cycle detection.

The cascade emitter is already a reducer over a run’s event log, though a degenerate one: it emits at most one follow-on event per confirmed cross-type write and performs no join. A multi-phase workflow is the same reducer with three parameters raised—an explicit plan

TABLE II
THE GUARANTEE REGISTER. EACH ROW NAMES A TEST EXECUTING
IN THE PROJECT’S ORDINARY TEST RUN.

	property
E-1	typed reference resolution is deterministic and total
E-2	a reference failing any of four checks is reported
E-3	anchors and versions survive resolution unchanged
E-4	concurrent editing sessions on one page converge
E-5	a mid-write crash rolls back every table together
E-6	the index is reproducible from documents alone
E-7	a stale base version is rejected, not silently merged
E-8	one event yields exactly one terminal run
E-9	a cascade terminates on depth or cycle detection
E-10	a workflow phase completes only at barrier quorum
E-11	non-native substrates refuse page-level writes
E-12	subscribed base pages cannot be written or forged
E-13	credentials never appear in the corpus
E-14	a remote read fails closed on kind mismatch
E-15	an unreachable endpoint degrades, never fabricates
E-16	an unresolvable write placeholder aborts upstream
E-17	source schema drift marks the binding degraded
E-18	a token minted for another tenant is refused

read from a workflow type, fan-out width greater than one, and a quorum barrier. We note this because fan-out orchestration is widely built as a second execution model beside a reactive one, and the model suggests it need not be: invoking a workflow is itself an inbox event, and the only thing distinguishing it from an externally captured one is which party captured it.

E. Types as Distributable Artifacts

Because types are documents, they are distributable. A pack is a versioned, read-only layer to which a tenant subscribes; local pages shadow base pages of the same name, promotion into the base layer passes a curator gate, and a pinned version does not move on its own. With the substrate corollary in play, a pack ships more than a schema: a customer-management pack carries types whose instances resolve against *the subscriber’s* registered endpoint, so vocabulary and integration arrive together and the subscriber supplies only a credential. We regard this as the model’s most consequential implication for practice, and the least explored.

VI. GUARANTEES

Table II states eighteen properties, each verified by an integration test executing in the project’s ordinary test command; none is disabled. The register is engineering discipline rather than validation of the model: it establishes that the implementation does what it says, not that the arrangement is preferable to another. We deliberately do not tabulate a comparison against vector stores here. Production retrieval systems are routinely wrapped in schemas, access control, versioning and transaction logs, so a column asserting they lack these properties would compare against a configuration rather than against any named system.

The eighteenth property records a limit of the present implementation rather than of the model: the specification

TABLE III
TIER-1 COST AGAINST CORPUS SIZE. TOKENS ARE `cl100k_base`;
LATENCY IS 30 SAMPLES PER COLUMN.

instances	10^2	10^3	10^4	10^5
Tier-1 tokens	744	744	744	744
Tier-1 p50 (ms)	0.74	0.74	0.76	0.81
Tier-1 p95 (ms)	0.88	0.91	0.80	1.16
whole corpus (tokens)	2.9k	29k	290k	2.9M
ratio avoided	$3.9\times$	$39\times$	$390\times$	$3898\times$

describes per-tenant handles within one process, these are not implemented, and the deployed isolation boundary is therefore the process. The property was added after an isolation defect in which authentication resolved a tenant from the presented token while the data path used the served tenant, and the two were never compared. The general observation is that a test suite can encode a defect as intended behaviour—here a foreign token succeeding was read as evidence that quota accounting was per-tenant—and that enumerating properties independently of the tests is what surfaces such cases.

VII. EVIDENCE

We report evidence for three claims. All measurements come from harnesses in the accompanying repository, each taken in a single session on one machine, since cross-session ratios are artifacts. No claim about agent behaviour is advanced, because no experiment here measures it.

A. Discovery Cost Is Invariant in Corpus Size

Table III measures cold-start cost over a synthetic tenant with 24 authored types and instance counts spanning 10^2 to 10^5 . Instances are inserted in bulk, as the quantity under test is a read.

Part of this is arithmetic rather than discovery, and we say so: the enumeration query selects on page type and cannot observe instances, so the payload could not have varied. Its value is as a guard—the harness asserts the equality, so if discovery ever begins to touch instances a test fails rather than a claim quietly becoming false.

The open question was latency, since discovery shares a table with every instance and the scan could have degraded. Across a thousand-fold increase in corpus size it moves from 0.74 to 0.81 ms at the median, roughly ten percent, while the material it declines to read grows to 2.9M tokens.

B. One Navigation Sequence, Five Substrates

Table IV reports the same entity described by five types, one per substrate, against a containerized relational database, an extracted binary file, a live remote service and a live upstream tool server. The same sequence—resolve, expand, traverse—runs against each.

All fifteen navigation steps succeed: one set of verbs, in one order, routed to a local file, a relational query, a chunked document and two live services. Two limits

TABLE IV

ONE ENTITY ON FIVE SUBSTRATES; 50 SAMPLES PER LATENCY CELL, ONE SESSION. SEARCH IS EACH TYPE'S *declared* CAPABILITY, NOT A PROBE.

	native	view	file	service	tool
navigation steps ok	3/3	3/3	3/3	3/3	3/3
declared search	hybrid	late	hybrid	none	none
expand p50 (ms)	3.8	9.4	5.8	6.7	6.5
expand p95 (ms)	4.7	10.2	6.9	8.3	7.7
payload (bytes)	380	760	1572	436	427

bound the claim. The caller is not blind to the substrate—the type segment of a reference names it, which is the design, since it is the type that declares the backend—so what is identical is the call shape rather than the caller's ignorance. And each instance is addressed by a handle obtained at creation, so this establishes navigation rather than discovery of an arbitrary entity.

Discoverability is read from each type's declaration rather than probed. A probe would have been invalid here: the fixture runs a zero-vector embedder, under which every vector is identical and search returns essentially the whole corpus. The declarations divide as the design specifies, with live proxies reporting no search capability. This row is therefore configuration inspection, not a behavioural measurement, and is included only to document where the abstraction is deliberately incomplete.

The latency ordering runs contrary to the intuition that materialized substrates answer locally in single-digit milliseconds while live proxies exhibit a network round trip. The relational view is instead the slowest column at 9.4ms, half again the cost of either proxy, because every read crosses into the external database while the proxies were answered by loopback services in the same process. Two consequences follow. The proxy figures are a floor excluding real network latency. And *materialized does not imply local*: a relational view is a live query behind a materialized interface, a distinction the design documentation blurred and this measurement does not. The ordering is descriptive of one session—50 warm calls, no independent repeats, single-digit millisecond gaps—but its direction is the opposite of the prediction.

C. Projection Produced No Duplicate Write Under Failure

Table V drains twenty events to quiescence under three configurations; Table VI reports recovery from process failure. Measurement uses a deterministic stand-in for the language model, so the figures describe the loop rather than inference latency.

Two candidate explanations for the throughput bound are unsupported in this fixture. The per-tenant governor admits 120 runs per minute over a fixed window, which averages two per second; raising it to 100,000 changed throughput not at all, so admission control is not the binding constraint here. Write contention is the second candidate, since all twenty events folded into one page behind a per-tenant write lock; giving each event its own

TABLE V

TWENTY EVENTS DRAINED TO QUIESCENCE, ONE SESSION. THE CLOCK STARTS AFTER CAPTURE. THE ARMS ARE NOT A CLEAN ISOLATION: LIFTING THE GOVERNOR ALSO CHANGES THE FAILURE PROFILE.

	governed	lifted	lifted
target instances	1	1	20
seconds to quiescence	9.17	9.66	9.87
events per second	2.18	2.07	2.03
ledger runs per event	1.25	1.75	1.75
stopped by a control	5	5	5
runs recorded failed	10	0	0
max folds of one event	1	1	1

TABLE VI

RECOVERY FROM PROCESS FAILURE, 100 TRIALS, KILL POINT SAMPLED FROM A DETERMINISTIC SEQUENCE IN [20, 100] MS. ONLY THE IN-FLIGHT ROW TESTS REPLAY.

kill landed	trials	converged
before the run started	0	—
while in flight	35	35
after the run completed	65	65
duplicate folds		0
never processed		0

target changed nothing either. Because the arms are not a clean isolation—lifting the governor also changes the failure profile—these are non-results rather than falsifications. What remains is an undecomposed residual of roughly 475ms per event; the harness times quiescence, not its components, so the serial per-run cost this suggests is a hypothesis. One side effect is measurable: lifting the governor moved runs recorded failed from ten to none at identical throughput, indicating it converts contention into retries rather than into delay.

The experiment requires a kill point that lands inside the write window, and verifying this is not optional: a first configuration sampling from a wider interval produced a hundred percent convergence in which *every* kill landed after the run had already completed, and therefore measured nothing. Instrumenting the kill phase and narrowing the interval placed 35 of 100 kills genuinely in flight. All 35 converged, with no duplicate projection and no event left unprocessed. The denominator is 35, not 100, and it bounds the duplicate rate loosely; a rare interleaving would not appear at this sample size. The result is also obtained with a deterministic stand-in for the language model, which removes the component most likely to lengthen the failure window, so it should not be read as an exactly-once guarantee in the distributed-systems sense.

D. Retrieval Mechanics (Not Evidence for the Position)

This subsection reports implementation context and bears on none of the claims above; a reader assessing the position may skip it. Over a public benchmark (SciFact, 5,183 documents, 300 queries, **bge-base-en-v1.5**): single-pass retrieval attains nDCG@10 of 0.723 and recall@100 of 0.976 at a median 141 ms, while a two-pass variant using

a 128-dimensional coarse pass attains 0.718 and 0.976 at 182ms. The optimization does not pay at this corpus size: 29% additional median latency and a third of throughput for identical recall. The measured latency is dominated by CPU query encoding rather than index search, making it a useful figure for an agent’s round trip and a misleading one for the index.

This benchmark cannot speak to the model’s central claim. A public IR corpus has no type structure, so a type-filtered query and an unfiltered one are the same query.

VIII. WHAT THIS DOES NOT ESTABLISH

Four experiments are specified in the accompanying repository and were not run. Each is stated because their absence bounds the claims above.

Whether an agent exploits the cheap tier. This is the behavioural half of the central claim and the most important gap. Table III shows the mechanism is available at fixed cost; it does not show that a language model reads descriptions, selects a type, and expands two records rather than twenty. The predecessor’s real-model run reached correct answers in 1–2^P tool calls at 12.4^P × lower token cost than a flat baseline on a twenty-page corpus, reported here as prototype context and not as a result for this system. A three-arm comparison against flat retrieval and whole-corpus prompting, at several scales, would settle it.

Whether typing improves retrieval quality. Separating the contribution of typing from that of metadata filtering requires a multi-type corpus with queries naming a type. No public benchmark we are aware of provides one. Constructing a synthetic corpus is possible and we regard it as the necessary next step; we did not attempt it here because a benchmark whose type partitioning is chosen by the authors of the system under test invites an obvious objection, and answering that objection requires a construction protocol agreed beyond one group. This bounds the present paper to a feasibility claim.

Whether the type count stays bounded. This is the falsifier: if authored types grow proportionally with generated records, discovery ceases to be cheap and the model fails. Establishing it requires corpus histories from long-running deployments in a form we can publish. We declined to substitute a curve drawn from example fixtures, since for this claim especially an unconvincing figure is worse than an acknowledged absence.

Whether storage consolidation cost retrieval quality. Thresholds were declared in advance before merging retrieval and relational state into one store; the labelled corpus against which they were declared was never versioned alongside the code and no longer exists. We record this as a process finding: an acceptance gate whose fixture is not versioned with the system is a gate still open at deployment.

Two negative results also bear recording. Lexical retrieval collapsed to 0.350^P nDCG on a duplicate-stress corpus where vector retrieval held at 0.933^P, contradicting the expectation that BM25 degrades gracefully under

near-duplication. And the encoder our own specification designates cannot be loaded by our own runtime, the inference library providing no path for that architecture; a specification and an implementation disagreed for weeks while each was individually correct when written.

IX. LIMITATIONS

The model is not universally applicable and several boundaries are sharp.

It requires a domain with vocabulary. Where every record is of one kind, discovery has nothing to enumerate and a schema’s authoring cost buys nothing. It requires people willing to author and curate types—precisely why type count stops growing, and equally why a team unwilling to do so obtains the cost without the benefit. Disclosure trades accuracy at the discovery step for tokens: the predecessor measured 13^P points of top-1 accuracy given up by description-only indexing, a poor trade for a corpus small enough to read whole.

The realization inherits further limits. Writes are serialized per tenant, which suits a knowledge base and not an event firehose. An embedded vector index inside a shared analytical file is not a dedicated vector service. Reads proxying remote calls inherit those calls’ tail latency, and live data is never indexed, so remote instances are reachable by reference and not by resemblance—yet Table IV shows materializing is not automatically faster, since a relational view was the slowest substrate measured. Multi-tenant isolation is presently a process boundary rather than an in-process one.

X. RELATED WORK

Agent memory. MemGPT [14] pages context into and out of a window, managing an untyped store rather than proposing to type it. Mem0 [15] and Zep [16] extract entities and relations into graphs automatically; the contrast with our position is deliberate, since a machine-inferred schema cannot be read in the morning, reviewed in a change request, or distributed as a versioned artifact. GraphRAG [17] summarizes at index time and answers global questions well, but the index is built rather than maintained: there is no write loop. A-Mem [18] and the memory stream of generative agents [19] organize episodic memory without a type system a domain expert would recognize as their own.

Progressive disclosure. Agent Skills [6] introduced disclosure for the procedural axis and stopped there. Our claim is that one small generalization—letting an instance declare its type—extends the mechanism to episodic memory, where in the domains we work in the volume, and therefore the cost, resides.

Federated access. Lakehouse catalogs [13], [20] and virtual query engines unify access to *data* without unifying the addressing of *knowledge*: they give one query language over many sources, whereas Section III-C gives one referent space over many substrates, of which relational data is one.

Event sourcing. The second duality descends directly from event sourcing [11]. What is new is that the projection

function is an agent, and its code is a document residing in the store it projects into.

XI. CONCLUSION

A skill is both schema and prompt; an instance is both record and document; an event is a fact and a callback its projection; and because the type declares the substrate, one referent space spans documents, relational views, extracted files and live services. The position advanced here is that co-locating these in one authored artifact is what allows a memory to grow without the enumeration an agent performs growing with it. The evidence establishes that the arrangement is realizable and that its enumeration cost is fixed across a thousand-fold increase in corpus size; it does not establish the behavioural benefit.

Of the three consequences of co-location, the substrate binding has the largest practical effect: it lets a typed memory reason over an organization’s data without first copying it, and makes a vocabulary and its integrations distributable as one versioned artifact.

The behavioural question remains open. Establishing that typed memory helps agents, as distinct from establishing that it is cheap to offer, requires multi-type corpora with type-naming queries, held-out tasks and ablations. A synthetic construction is feasible and is the next step we intend; its weakness is that a partitioning chosen by the authors of the system under test is contestable, which argues for publishing the generator and the tasks separately from any system that consumes them.

ACKNOWLEDGMENT

The design proposed here was first sketched in an earlier draft [21] and evaluated on a prototype; figures carried forward from that work are marked throughout.

REFERENCES

- [1] A. Hogan, E. Blomqvist, M. Cochez, C. d’Amato, G. de Melo, C. Gutierrez *et al.*, “Knowledge graphs,” *ACM Computing Surveys*, vol. 54, no. 4, pp. 1–37, 2021.
- [2] M. Krötzsch, D. Vrandečić, M. Völkel, H. Haller, and R. Studer, “Semantic Wikipedia,” *Journal of Web Semantics*, vol. 5, no. 4, pp. 251–261, 2007.
- [3] D. Vrandečić and M. Krötzsch, “Wikidata: A free collaborative knowledgebase,” *Communications of the ACM*, vol. 57, no. 10, pp. 78–85, 2014.
- [4] R. V. Guha, D. Brickley, and S. Macbeth, “Schema.org: Evolution of structured data on the web,” *Communications of the ACM*, vol. 59, no. 2, pp. 44–51, 2016.
- [5] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language models can teach themselves to use tools,” *NeurIPS*, 2023.
- [6] Anthropic, “Equipping agents for the real world with Agent Skills,” <https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>, 2025, engineering blog post.
- [7] G. Xiao, L. Ding, B. Cogrel, and D. Calvanese, “Virtual knowledge graphs: An overview of systems and use cases,” *Data Intelligence*, vol. 1, no. 3, pp. 201–223, 2019.
- [8] M. Franklin, A. Halevy, and D. Maier, “From databases to dataspace: A new abstraction for information management,” in *SIGMOD Record*, vol. 34, no. 4, 2005, pp. 27–33.
- [9] C. Bizer, T. Heath, and T. Berners-Lee, “Linked data — the story so far,” *Int. J. Semantic Web Inf. Syst.*, vol. 5, no. 3, pp. 1–22, 2009.
- [10] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “Lost in the middle: How language models use long contexts,” *TACL*, vol. 12, pp. 157–173, 2024.
- [11] M. Fowler, *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002, event Sourcing pattern.
- [12] M. Raasveldt and H. Mühleisen, “DuckDB: an embeddable analytical database,” in *Proc. SIGMOD*, 2019, pp. 1981–1984.
- [13] DuckDB Labs, “DuckLake: SQL as a lakehouse format,” <https://ducklake.select/>, 2025.
- [14] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, “MemGPT: Towards LLMs as operating systems,” *arXiv:2310.08560*, 2023.
- [15] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav, “Mem0: Building production-ready AI agents with scalable long-term memory,” *arXiv:2504.19413*, 2025.
- [16] P. Rasmussen, P. Paliychuk, T. Beauvais, J. Ryan, and D. Chalef, “Zep: A temporal knowledge graph architecture for agent memory,” *arXiv:2501.13956*, 2025.
- [17] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, “From local to global: A graph RAG approach to query-focused summarization,” *arXiv:2404.16130*, 2024.
- [18] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang, “A-Mem: Agentic memory for LLM agents,” *arXiv:2502.12110*, 2025.
- [19] J. S. Park, J. C. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, “Generative agents: Interactive simulacra of human behavior,” in *Proc. UIST*, 2023.
- [20] Apache Software Foundation, “Apache Iceberg: The open table format for analytic datasets,” <https://iceberg.apache.org/>, 2024.
- [21] J. Roskopf, “Progressive disclosure of agent memory through a typed skill-instance unification,” 2026, research draft, 2026-05-16.